

Aplikacja budżetowa / finanse osobiste z dashboard'em

1. Zdanie projektowe

Aplikacja ma pomóc użytkownikowi zarządzającemu finansami osobistymi w sytuacji planowania przyszłych wydatków podjąć decyzję o wykonaniu, odłożeniu lub ograniczeniu planowanego zakupu, na podstawie historii transakcji, aktualnego stanu budżetu, celów oszczędnościowych oraz prognozowanych wydatków i przychodów.

Aplikacja nie będzie wyłącznie prezentowała wykresy. Będzie również oferować funkcjonalności, które ułatwią podejmowanie decyzji finansowych.

2. Główne funkcjonalności:

Podstawowy moduł zarządzania:

- rejestracja i logowanie użytkowników
- zarządzanie profilem
- przechowywanie danych oraz finansów

Moduł zarządzania finansami:

- dodawanie przychodów i wydatków
- przypisywanie transakcji do kategorii
- edycja i usuwanie zapisanych transakcji
- przechowywanie historii operacji finansowych

Moduł budżetów i celów finansowych:

- tworzenie budżetów dla wybranych kategorii wydatków

- definiowanie celów krótko- i długoterminowych
- monitorowanie postępu realizacji celów

Moduł analityczny i dashboard:

- prezentacja przychodów i wydatków w formie wykresów
- identyfikacja największych źródeł kosztów
- porównywanie aktualnych wydatków z poprzednimi okresami

Moduł kontrolny:

- ~~— wykrywanie nietypowych wzrostów wydatków~~
- ~~— wysyłanie powiadomień e-mail lub w aplikacji~~

Moduł predykcji i rekomendacji

- ~~— prognozowanie stanu środków finansowych na podstawie historii wydatków~~
- ~~— przewidywanie ryzyka przekroczenia budżetu~~
- ~~— uwzględnianie cyklicznych oraz sezonowych wydatków~~
- ~~— generowanie rekomendacji dotyczących oszczędzania~~

W trakcie prac nad projektem koncepcja modułu kontrolnego oraz modułu predykcji i rekomendacji uległa zmianie. Zamiast dwóch oddzielnych modułów powstał jeden — Moduł doradztwa finansowego oparty na LLM, który łączy w sobie obie funkcjonalności w bardziej zaawansowanej formie:

Moduł doradztwa finansowego (LLM):

- prognozowanie wydatków na podstawie regresji liniowej na historii transakcji
- generowanie spersonalizowanych rekomendacji oszczędnościowych przez model językowy (Ollama / Llama 3.2)
- respektowanie preferencji użytkownika — kategorie oznaczone jako „chronione” nie są brane pod uwagę przy sugestjach cięć
- optymalizacja budżetu z limitem maksymalnego cięcia 30% per kategoria
- streamowanie odpowiedzi w czasie rzeczywistym (SSE)

- predefiniowane analizy: pełna analiza, identyfikacja przepływów, plan oszczędzania, prognoza, porady

Uwaga dotycząca regulacji finansowych

Aplikacja nie stanowi usługi doradztwa finansowego w rozumieniu ustawy o obrocie instrumentami finansowymi. Zgodnie z przepisami nadzorowanymi przez Komisję Nadzoru Finansowego (KNF), działalność wymaga licencji doradcy inwestycyjnego wyłącznie wtedy, gdy spełnione są łącznie następujące warunki:

- aplikacja wydaje rekomendacje dotyczące konkretnych instrumentów finansowych (akcje, obligacje, fundusze inwestycyjne)
- rekomendacje są kierowane do zindywidualizowanego odbiorcy w oparciu o jego sytuację finansową
- usługa ma charakter odpłatny lub jest świadczona w ramach działalności regulowanej

Aplikacja nie spełnia żadnego z tych warunków — nie rekomenduje instrumentów finansowych, nie sugeruje inwestycji, a jedynie analizuje wydatki osobiste użytkownika i proponuje optymalizację budżetu domowego. Jest to narzędzie do zarządzania finansami osobistymi.

3. Aktorzy i tryby pracy

~~Aplikacja będzie ukierunkowana na użytkowników. Dodatkowo system będzie posiadał rolę administratora odpowiedzialnego za utrzymanie aplikacji.~~

Pierwotna koncepcja zakładała osobną rolę administratora z dedykowanym panelem — w trakcie realizacji projektu podjęto decyzję o zastąpieniu panelu administracyjnego narzędziami observability (Prometheus + Grafana), które zapewniają monitoring systemu bez konieczności budowania osobnego interfejsu.

Rola	Typowa decyzja	Narzędzie	Pytanie kontrolne
User	Zarządzanie finansami	Aplikacja webowa	Czy mogę pozwolić sobie na planowany wydatek i czy realizuję swoje cele finansowe?
Admin/DevOps	Monitoring systemu	Grafana Prometheus	Czy system działa poprawnie? Czy występują anomalie w metrykach?

4. Główne widoki

Route	Cel widoku	Kluczowe komponenty
/login	Logowanie i rejestracja	Formularz logowania z przełącznikiem na rejestrację
/	Strona główna (po zalogowaniu)	Kafelki nawigacyjne do poszczególnych modułów
/dashboard	Analiza sytuacji finansowej	Konfigurowalne wykresy: przychody vs wydatki, wydatki wg kategorii, trendy, porównania, realizacja budżetów, predykcja AI
/transactions	Zarządzanie transakcjami	Lista transakcji z paginacją i filtrowaniem, formularz dodawania/edycji
/categories	Zarządzanie kategoriami	Lista kategorii z podziałem na wydatki/przychody, oznaczanie kategorii jako chronionych
/budgets	Zarządzanie budżetem	Limity wydatków per kategoria, procent wykorzystania, rozwijana lista transakcji per budżet
/goals	Cele finansowe	Lista celów z progress barami, wpłaty na cele
/advisor	Doradca finansowy AI	Predykcje wydatków, predefiniowane analizy, własne pytania, streamowanie odpowiedzi LLM

Monitoring (zastępuje panel admina)

Adres	Narzędzie	Cel
localhost:3001	Grafana	Dashboardy z wykresami metryk, alerty
localhost:9090	Prometheus	Zapytania o metryki, status targetów
localhost:3000/metrics	Endpoint metryk	Metryki HTTP, biznesowe, Node.js
localhost:3000/api/docs	Swagger	Interaktywna dokumentacja API

5. Interakcje krytyczne

Dla widoku / (Strona główna):

1. User loguje się — system weryfikuje JWT, pobiera dane użytkownika i przekierowuje na stronę główną z kafelkami nawigacyjnymi.
2. User klika kafelek (np. „Dashboard”) — aplikacja nawiguje do odpowiedniego widoku bez przeładowania strony (React Router, SPA).
3. User klika „Wyloguj” — system usuwa httpOnly cookie z tokenem JWT i przekierowuje na stronę logowania.

Dla widoku /dashboard:

1. User wchodzi na /dashboard — system równolegle pobiera podsumowanie miesięczne, wydatki wg kategorii oraz trend z ostatnich 6 miesięcy i wyświetla domyślne wykresy (podsumowanie, bar chart, pie chart).
2. User klika „Dodaj wykres” — system wyświetla panel z 8 dostępnymi widżetami. User toggleuje widoczność poszczególnych wykresów, zmiany są natychmiastowe bez przeładowania strony.
3. User włącza widżet „Porównanie miesięcy” — system pobiera dane z endpointu /analytics/compare (lazy loading, dane ładowane dopiero po włączeniu widżetu) i wyświetla różnice procentowe względem poprzedniego miesiąca.

4. User włącza widżet „Predykcja AI” — system pobiera prognozę z /advisor/quick (regresja liniowa na historii wydatków) i wyświetla przewidywaną kwotę wraz z trendami per kategoria.

Dla widoku /transactions:

1. User wchodzi na /transactions — system pobiera pierwszą stronę transakcji (limit 10, sortowanie po dacie malejąco) oraz listę kategorii.
2. User filtruje po typie (Wydatki/Przychody) — system resetuje paginację i pobiera przefiltrowane dane. Filtr aktywny jest wizualnie wyróżniony.
3. User klika „Dodaj transakcję” — system wyświetla formularz inline. Lista kategorii w selekcji filtruje się automatycznie na podstawie wybranego typu (wydatek → kategorie wydatkowe, przychód → kategorie przychodowe).
4. User edytuje transakcję — system wypełnia formularz danymi istniejącej transakcji. Po zapisie invaliduje cache transakcji oraz analityki (TanStack Query), aby dashboard odzwierciedlał aktualne dane.
5. User usuwa transakcję — system usuwa rekord i invaliduje powiązane cache bez przeładowania strony.

Dla widoku /categories:

1. User wchodzi na /categories — system pobiera kategorie z MongoDB i wyświetla je w dwóch kolumnach: wydatki i przychody.
2. User tworzy kategorię — podaje nazwę, typ i kolor. System zapisuje do MongoDB i odświeża listę.
3. User oznacza kategorię jako „chronioną” (ikona tarczy) — system aktualizuje flagę isProtected w MongoDB. Doradca AI respektuje tę flagę i nie sugeruje cięć w tej kategorii.
4. User usuwa kategorię — system usuwa dokument z MongoDB. Istniejące transakcje zachowują categoryId, ale wyświetlają surowe ID zamiast nazwy.

Dla widoku /budgets:

1. User wchodzi na /budgets — system pobiera podsumowanie budżetów dla wybranego miesiąca i roku, łącznie z obliczonym procentem wykorzystania (wydatki / limit).
2. User zmienia miesiąc lub rok w selekcje — system pobiera budżety dla nowego okresu bez przeładowania strony.
3. User przekracza budżet — system oznacza progress bar na czerwono, wyświetla procent powyżej 100% oraz komunikat o kwocie przekroczenia. Nie blokuje dalszej pracy.
4. User klika na budżet — system rozwija sekcję ze szczegółami i pobiera listę transakcji powiązanych z tą kategorią w danym miesiącu (lazy loading). Ponowne kliknięcie zwija sekcję.
5. User dodaje nowy budżet — wybiera kategorię wydatkową i kwotę limitu. System tworzy budżet i przelicza podsumowanie.

Dla widoku /goals:

1. User wchodzi na /goals — system pobiera cele finansowe i wyświetla je jako karty z progress barami, kwotami i liczbą dni do deadline'u.
2. User tworzy cel — podaje nazwę, kwotę docelową i termin. System tworzy rekord z `currentAmount = 0`.
3. User klika „Wpłać na cel” — system wyświetla pole kwoty inline. Po wpłacie aktualizuje `currentAmount` i przelicza procent realizacji.
4. Cel po terminie — system wyświetla liczbę dni po terminie na czerwono, ale nie blokuje dalszych wpłat.

Dla widoku /advisor:

1. User wchodzi na /advisor — system pobiera szybką analizę z /advisor/quick (predykcja wydatków, wydatki per kategoria z trendami, liczba chronionych kategorii) i wyświetla karty podsumowania.

2. User klika predefiniowany kafelek analizy (np. „Gdzie przepłacam?”) — system otwiera połączenie SSE do /advisor/advise z odpowiednim pytaniem. Podczas oczekiwania na odpowiedź wyświetlane są pulsujące kropki.
3. LLM zaczyna generować odpowiedź — system streamuje tekst token po tokenie z efektem migającego kursora. User widzi „piszącą się” rekomendację w czasie rzeczywistym.
4. User wpisuje cel oszczędności (np. 500 PLN) i zadaje własne pytanie — system przekazuje oba parametry do backendu. Backend oblicza plan oszczędności (regresja liniowa + optymalizacja z wyłączeniem chronionych kategorii), pakuje wyniki w prompt i wysyła do LLM.
5. Ollama nie jest uruchomiona — system wyświetla komunikat o błędzie zamiast streamowanej odpowiedzi, bez zawieszenia interfejsu

6. Model danych frontendowych

```
TS frontLts > Category
1 // Modele danych używane po stronie frontendu
2 // Odpowiadają strukturom zwracanym przez REST API
3
4 // PostgreSQL – dane relacyjne (Prisma ORM)
5
6 export type User = {
7   id: string;
8   email: string;
9   name: string;
10 }
11
12 export type Transaction = {
13   id: string;
14   userId: string;
15   amount: number;
16   type: "income" | "expense";
17   description: string;
18   date: string;
19   categoryId: string; // referencja do kategorii w MongoDB
20   createdAt: string;
21 }
22
23 export type Budget = {
24   id: string;
25   userId: string;
26   categoryId: string; // referencja do kategorii w MongoDB
27   amount: number;
28   month: number;
29   year: number;
30 }
31
```



```

42 // MongoDB – dane dokumentowe (Mongoose ODM)
43
44 export type Category = {
45   _id: string;           // MongoDB ObjectId
46   userId: string;
47   name: string;
48   color: string;
49   type: "income" | "expense";
50   isProtected: boolean; // flaga dla doradcy AI
51 }

```

Wy tłumaczenie:

W PostgreSQL kategorie wymagałyby sztywnej tabeli z predefiniowanymi kolumnami. Każda zmiana struktury (dodanie nowego pola, np. ikona, priorytet, limit wydatków per kategoria) wymagałaby migracji bazy danych. Wszystkich użytkowników obowiązywałby ten sam schemat.

W MongoDB każdy użytkownik może mieć kategorie o dowolnej strukturze — baza się do tego dostosowuje, a nie odwrotnie.

7. Stos technologiczny

Warstwa	Technologia
Frontend	React 19 + Vite + TypeScript
Stylowanie	Tailwind CSS 4
Wykresy	Recharts
State management	TanStack Query + Zustand
Routing	React Router 7
Formularze	React Hook Form + Zod
Backend	Node.js + Express 5 + TypeScript
ORM (SQL)	Prisma 5
ODM (NoSQL)	Mongoose 9

Warstwa	Technologia
Autentykacja	JWT (jsonwebtoken) + bcrypt
LLM	Ollama (Llama 3.2, 3B)
Obliczenia	simple-statistics
Dokumentacja API	Swagger (swagger-jsdoc + swagger-ui-express)
Metryki	prom-client
Baza SQL	PostgreSQL 16
Baza dokumentowa	MongoDB 7
Monitoring	Prometheus + Grafana
CI/CD	GitHub Actions
Konteneryzacja	Docker + docker-compose
Linting	Biome
Testy	Jest + ts-jest
Monorepo	pnpm workspaces

8. ADR

Pole	Treść
Decyzja	PostgreSQL jako główna baza danych
Kontekst	Aplikacja przechowuje dane finansowe użytkowników: transakcje, budżety miesięczne, cele oszczędnościowe. Dane mają wyraźne relacje — każda transakcja należy do użytkownika, budżet łączy użytkownika z kategorią w konkretnym miesiącu, cel ma właściciela. Wymagana jest spójność danych (nie może powstać transakcja bez użytkownika) oraz agregacje (suma wydatków per kategoria, porównania miesięczne)
Alternatywy	MongoDB (dokumentowa) — elastyczny schemat, ale brak natywnych JOINów utrudnia agregacje relacyjne (budżet vs wydatki w kategorii). Wymuszałby denormalizację lub wielokrotne zapytania. SQLite — zero konfiguracji, ale brak obsługi współbieżności (jeden zapis blokuje bazę) i problematyczne w kontenerach Docker.

	TimescaleDB (szeregi czasowe) — optymalne dla danych z timestampem, ale transakcje finansowe to kilka zapisów dziennie, nie tysiące na sekundę. Overhead konfiguracji nieuzasadniony przy tej skali.
Uzasadnienie	Dane finansowe zmieniają się umiarkowanie: budżety i cele rzadko (raz w miesiącu), transakcje kilka razy dziennie/tygodniowo, predykcje generowane na żądanie. Kluczowe są relacje i agregacje — PostgreSQL obsługuje je natywnie (GROUP BY, JOINy, UNIQUE constraints).
Trade-offy	Sztwywny schemat wymaga migracji przy każdej zmianie struktury. Mniej elastyczny niż MongoDB dla danych o zmiennym kształcie (dlatego kategorie przechowujemy osobno w MongoDB).

Pole	Treść
Decyzja	MongoDB jako baza dokumentowa dla kategorii użytkowników
Kontekst	Użytkownicy definiują własne kategorie wydatków i przychodów. Każdy użytkownik może mieć inną strukturę kategorii — jeden potrzebuje tylko nazwy i koloru, inny chce podkategorie, limity miesięczne, notatki czy budżety sezonowe. Kategorie zawierają flagę isProtected, która informuje doradcę AI, że dana kategoria nie powinna być brana pod uwagę przy sugestjach cięć.
Alternatywy	PostgreSQL (osobna tabela) — wymagałby kolumn na każde możliwe pole (subcategories, monthlyLimit, seasonalBudget, notes) lub kolumny JSONB, co niweluje zalety relacyjnej bazy. Każde rozszerzenie struktury wymaga migracji wpływającej na wszystkich użytkowników. Plik JSON na dysku — brak indeksowania, wyszukiwania, współbieżności. Redis — idealny jako cache, ale nie jako primary storage (dane giną po restarcie bez konfiguracji persystencji).
Uzasadnienie	MongoDB pozwala każdemu dokumentowi mieć inną strukturę bez migracji. Użytkownik A ma kategorie z 4 polami, użytkownik B z 8 — baza akceptuje oba warianty. Mongoose ODM zapewnia walidację bazową (wymagane pola: userId, name, type) przy jednoczesnej elastyczności dla pól opcjonalnych. Indeks compound (userId + name) zapewnia unikalność kategorii per użytkownik.
Trade-offy	Dwie bazy danych zwiększają złożoność infrastruktury (dwa kontenery w docker-compose, dwa połączenia do zarządzania). Brak transakcji bazodanowych między PostgreSQL a MongoDB — nie można atomowo usunąć kategorii i powiązanych transakcji. Referencja categoryId w PostgreSQL to zwykły string, nie foreign key — spójność referencyjna leży po stronie aplikacji.

Pole	Treść
Decyzja	SSE (Server-Sent Events) jako mechanizm komunikacji real-time dla doradcy AI
Kontekst	Moduł doradztwa finansowego wywołuje lokalny model LLM (Ollama / Llama 3.2), który generuje odpowiedź token po tokenie. Czas generowania pełnej odpowiedzi to 5-30 sekund. Użytkownik musi widzieć postęp generowania w czasie rzeczywistym, a nie czekać na pustym ekranie
Alternatywy	Synchroniczne API — jeden request, czekanie 5-30s na pełną odpowiedź. Ryzyko timeout (domyślne 30s w wielu proxy). Użytkownik nie wie czy apka działa czy się zawiesiła. Każde odświeżenie strony restartuje obliczenia. Job API (async z kolejką) — request tworzy zadanie, frontend polluje po status, osobny endpoint na wynik. Wymaga kolejki (Redis + Bull/BullMQ), zarządzania stanem jobów (pending/running/completed/failed), trzech endpointów zamiast jednego., overkill gdy jedynym konsumentem jest jeden użytkownik czekający na odpowiedź. WebSocket — dwukierunkowa komunikacja. Złożoność utrzymania połączenia, problemy z proxy/firewallami, load balancerami. Niepotrzebna komunikacja klient→serwer skoro user tylko odbiera odpowiedź.
Uzasadnienie	SSE działa przez standardowy HTTP GET — zero problemów z proxy, CORS, firewallami. Express natywnie obsługuje SSE przez res.write(). Frontend używa natywnego fetch + ReadableStream. Efekt „pisania” identyczny jak w ChatGPT. Jedno połączenie, zero dodatkowej infrastruktury, zero nowych zależności. Połączenie zamyka się automatycznie po zakończeniu generowania.
Trade-offy	Komunikacja jednokierunkowa — user nie może przerwać generowania w połowie (musiałby odświeżyć stronę). Brak natywnego HTTP caching (każde zapytanie do doradcy jest unikalne, więc caching i tak nie miałby sensu). Brak automatycznego reconnectu przy zerwaniu połączenia (akceptowalne — user może kliknąć ponownie).

Pole	Treść
Decyzja	Rozdzielenie obliczeń matematycznych (backend) od interpretacji (LLM)
Kontekst	Moduł doradztwa musi wykonać dwa rodzaje zadań: (1) precyzyjne obliczenia — prognoza wydatków, analiza trendów, optymalizacja planu oszczędności z limitem 30% cięcia per kategoria i wykluczeniem chronionych kategorii, (2) komunikacja wyników — sformułowanie rekomendacji w naturalnym, zrozumiałym języku dopasowanym do sytuacji użytkownika.
Alternatywy	LLM liczy i interpretuje — model dostaje surowe transakcje i sam oblicza sumy, średnie, trendy, prognozy. Problem: LLM-y są zawodne w precyzyjnej arytmetyce (hallucynacje liczbowe), nie gwarantują powtarzalności wyników, nie respektują deterministycznie reguł biznesowych (np. limit 30% cięcia, ochrona kategorii). Tylko obliczenia bez LLM — backend generuje suchy raport liczbowy. Problem: użytkownik dostaje tabelę liczb bez kontekstu, bez wyjaśnienia co one oznaczają, bez praktycznych porad.
Uzasadnienie	Backend oblicza z gwarancją precyzji: regresja liniowa (simple-statistics), agregacje SQL (Prisma groupBy), optymalizacja budżetu (deterministyczny algorytm). LLM interpretuje te liczby i formułuje ludzką rekomendację. Dzięki temu: liczby są zawsze poprawne (backend), rekomendacje są zawsze czytelne (LLM), reguły biznesowe są zawsze respektowane (backend wymusza je przed wystaniem do LLM)
Trade-offy	Dwie warstwy do utrzymania zamiast jednej. Prompt musi być starannie skonstruowany, żeby LLM poprawnie interpretował dane (ale nie przeliczał ich). LLM może wygenerować poradę sprzeczną z danymi (np. „wydajesz mało na jedzenie” gdy dane mówią inaczej) — minimalizujemy to przez szczegółowy prompt z explicite podanymi liczbami

Pole	Treść
Decyzja	Prometheus + Grafana zamiast dedykowanego panelu administratora
Kontekst	Pierwotna koncepcja zakładała panel /admin z widokiem na bazę danych i monitoring aplikacji. Potrzebny jest wgląd w stan systemu: ile requestów obsługuje backend, jakie są czasy odpowiedzi, ile pamięci zużywa proces, ile transakcji tworzą użytkownicy.
Alternatywy	Dedykowany panel admin — osobny widok w React z tabelami użytkowników, transakcji, logami. Wymaga: osobnych endpointów API, roli admin w systemie uprawnień, dodatkowych stron frontendowych, autoryzacji per endpoint. Tygodnie pracy na funkcjonalność którą Grafana daje out-of-the-box. Logi w konsoli — console.log w backendzie, przeglądanie w terminalu. Brak historii, brak wykresów, brak alertów. Nie skaluje się. Datadog/New Relic (SaaS) — zaawansowane, ale płacone i wymaga wysyłania danych do zewnętrznego serwisu.
Uzasadnienie	Prometheus scrapuje endpoint /metrics co 5 sekund — zero kodu po stronie backendu poza definicją metryk (prom-client). Grafana daje konfigurowalne dashboardy z wykresami, alertami, filtrami czasowymi. Obie usługi są darmowe, open-source, uruchamiane jako kontenery w docker-compose. Metryki biznesowe (transactions_created_total, advisor_requests_total) dodane w kilku liniach kodu. Strukturalne logi JSON (timestamp, level, method, path, status, duration_ms) dają pełny audit trail.
Trade-offy	Prometheus i Grafana to dodatkowe dwa kontenery w docker-compose. Konfiguracja dashboardów w Grafanie jest ręczna (nie jest wersjonowana w repo). Brak bezpośredniego widoku na dane w bazie — ale to celowe (admin nie powinien przeglądać transakcji użytkowników przez UI).

Pole	Treść
Decyzja	Monorepo z pnpm workspaces
Kontekst	Projekt składa się z trzech paczek: backend (Express), frontend (React), shared (współdzielone typy TypeScript). Potrzebna jest strategia organizacji kodu.
Alternatywy	Multi-repo — osobne repozytoria per paczka, niezależne releasy. Synchronizowanie typów między frontem a backiem wymaga publikowania paczki shared do npm lub kopiowania plików ręcznie. Osobne CI/CD per repo. Jeden folder bez workspace'ów — backend i frontend w jednym repo, ale bez formalnej separacji. Wspólny package.json ze wszystkimi zależnościami (React + Express + Prisma + Mongoose razem).
Uzasadnienie	npm workspaces daje separację zależności (frontend nie instaluje Prisma, backend nie instaluje React) przy jednoczesnym współdzieleniu typów. Zmiana interfejsu Transaction w shared powoduje natychmiastowy błąd TypeScript w obu paczkach — bez publikowania, bez kopiowania. Jeden git clone, jeden docker-compose, jeden CI pipeline. Dla solo dewelopera multi-repo to niepotrzebny overhead administracyjny.

Trade-offy	Lockfile (pnpm-lock.yaml) jest wspólny i duży. Problemy z jedną paczką mogą blokować instalację w drugiej. Pnpm workspaces ma mniej dokumentacji niż Nx czy Turborepo (ale nie potrzebujemy ich funkcjonalności).
-------------------	---